

Comprehensive Evaluation of Quantization Methods for Edge LLM Deployment with RAG Focus

Zahraa Selim, Menna Hamed, Wesam Ahmed,
Sohyla Said, Sara Basheer, Rami Zewail

*Computer Science and Engineering Department
Egypt-Japan University of Science and Technology (E-JUST)*

Alexandria, Egypt

{zahraa.selim, menna.hamed, rami.zewail}@ejust.edu.eg

December 29, 2025

1 Methodology

1.1 Overview

This study conducts a comprehensive evaluation of post-training quantization techniques applied to the Mistral-7B model, with particular focus on Retrieval-Augmented Generation (RAG) capabilities. Our experimental protocol follows a two-phase approach: (1) quantize the base model on high-memory hardware, and (2) evaluate compressed models on resource-constrained edge hardware to simulate realistic deployment scenarios. All evaluations assess three dimensions: computational efficiency, general task performance, and RAG-specific capabilities.

1.2 Hardware and Software Environment

1.2.1 Quantization Environment

Model quantization is performed on cloud-based GPU instances with sufficient memory to load and process the full-precision model:

- **Platform:** Kaggle
- **GPU:** NVIDIA Tesla P100 (16GB VRAM)

The P100 provides sufficient memory bandwidth and compute capacity for the calibration-based quantization algorithms (GPTQ, AWQ), which require multiple forward passes through calibration datasets to compute optimal quantization parameters. This separation of quantization and evaluation environments reflects realistic deployment workflows where model compression is performed once on powerful hardware before deployment to edge devices.

1.2.2 Evaluation Environment

All performance evaluations are conducted on edge-class hardware to assess real-world deployment viability:

- **Platforms:** Kaggle and Google Colab free-tier instances

- **GPU:** NVIDIA Tesla T4 (16GB VRAM)

The Tesla T4 GPU represents a common edge deployment target, offering a balance between performance and power efficiency (70W TDP). Its 16GB memory constraint necessitates model compression for serving 7B parameter models, making it an ideal testbed for evaluating quantization effectiveness.

1.3 Base Model

We select **Mistral-7B-v0.1** [?] as our baseline model for the following reasons:

- **Architecture:** Modern transformer with Grouped Query Attention (GQA) and Sliding Window Attention (SWA), representative of current LLM designs
- **Scale:** 7.24 billion parameters—large enough to exhibit quantization challenges while remaining evaluable on edge hardware
- **Performance:** Strong baseline capabilities across reasoning, knowledge, and generation tasks
- **Format:** FP16 baseline requiring 13.49 GB storage, necessitating compression for edge deployment

The FP16 baseline model serves as our reference point for all performance comparisons, with measurements conducted under identical evaluation protocols to ensure fair comparison.

1.4 Compression Techniques

We evaluate four state-of-the-art post-training quantization (PTQ) methods, all targeting 4-bit precision to achieve approximately $3.6\times$ compression:

1.4.1 NF4 (NormalFloat 4-bit)

NF4 employs an information-theoretically optimal quantization data type designed for normally distributed weights [?]. The method leverages the empirical observation that pre-trained neural network weights follow approximately Gaussian distributions.

Technical Approach:

NF4 constructs a 4-bit data type by quantizing weight distributions into 16 bins positioned at the quantiles of a standard normal distribution $\mathcal{N}(0, 1)$. For a weight w drawn from $\mathcal{N}(0, \sigma^2)$, the quantization maps w to the nearest quantile bin:

$$\hat{w} = \text{NF4}(w) = \sigma \cdot q_i, \quad \text{where } i = \arg \min_j |w - \sigma \cdot q_j| \quad (1)$$

where q_i are the 16 quantile values: $\{-1.0, -0.6962, -0.5251, \dots, 0.5251, 0.6962, 1.0\}$.

Double Quantization:

To further reduce memory overhead, NF4 quantizes the quantization constants (scales σ) themselves using blockwise FP8 quantization. For a tensor divided into blocks of size 64, each block’s scale is stored in 8-bit floating point rather than FP32, reducing quantization metadata by 75%.

Configuration:

- Quantization type: NF4 (16 quantile bins)
- Double quantization: Enabled (FP8 scales)
- Compute dtype: FP16 (dequantized computation)
- Block size: 64 (default bitsandbytes configuration)

- On-the-fly quantization: Model quantized during loading

Implementation:

Listing 1: NF4 Quantization Configuration

```

1 from transformers import BitsAndBytesConfig, AutoModelForCausalLM
2 import torch
3
4 nf4_config = BitsAndBytesConfig(
5     load_in_4bit=True,
6     bnb_4bit_quant_type="nf4",
7     bnb_4bit_use_double_quant=True,
8     bnb_4bit_compute_dtype=torch.float16
9 )
10
11 model = AutoModelForCausalLM.from_pretrained(
12     "mistralai/Mistral-7B-v0.1",
13     quantization_config=nf4_config,
14     device_map="auto"
15 )

```

1.4.2 GPTQ (Generative Pre-trained Transformer Quantization)

GPTQ is a layer-wise quantization algorithm that minimizes reconstruction error using second-order information [?]. The method reformulates optimal weight quantization as a series of layerwise least-squares problems.

Mathematical Formulation:

For each layer’s weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, GPTQ solves:

$$\min_{\hat{\mathbf{W}}} \|\mathbf{W}\mathbf{X} - \hat{\mathbf{W}}\mathbf{X}\|_F^2 \quad (2)$$

where $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$ represents n calibration samples, and $\hat{\mathbf{W}}$ is the quantized weight matrix.

GPTQ employs a greedy coordinate descent strategy inspired by Optimal Brain Quantization (OBQ) [?], quantizing weights column-by-column while compensating for quantization errors in subsequent weights. The key innovation is using the Hessian matrix $\mathbf{H} = \frac{2}{n}\mathbf{X}\mathbf{X}^T$ to guide error compensation:

$$w_{q'} \leftarrow w_{q'} - \delta_q \cdot \frac{H_{qq'}}{H_{qq}} \quad \forall q' > q \quad (3)$$

where $\delta_q = w_q - \hat{w}_q$ is the quantization error for weight w_q .

Group-wise Quantization:

GPTQ applies quantization in groups to balance precision and compression. Each group of $g = 128$ consecutive weights shares quantization parameters (scale s and zero-point z):

$$\hat{w}_i = \left\lfloor \frac{w_i - z \cdot s}{s} \right\rfloor, \quad s = \frac{\max(w_g) - \min(w_g)}{2^b - 1} \quad (4)$$

where $b = 4$ bits and $\lfloor \cdot \rfloor$ denotes rounding to nearest integer.

Quantization Process:

We quantized Mistral-7B-v0.1 using AutoGPTQ on Tesla P100 GPUs:

- **Base model:** mistralai/Mistral-7B-v0.1 (7.24B parameters)
- **Precision:** 4-bit integer quantization

- **Group size:** 128 (weights per quantization group)
- **Calibration dataset:** [To be specified]
- **Calibration samples:** 128 sequences of maximum length 512 tokens
- **Damping factor:** 0.01 (Hessian regularization: $\mathbf{H} + \lambda \mathbf{I}$)
- **Memory optimization:** CPU offloading enabled (`cache_examples_on_gpu=False`)
- **Quantization time:** 22 minutes on P100 GPU

Implementation:

Listing 2: GPTQ Quantization Process

```

1 from auto_gptq import AutoGPTQForCausalLM, BaseQuantizeConfig
2 from transformers import AutoTokenizer
3
4 # Configure quantization
5 quant_config = BaseQuantizeConfig(
6     bits=4,
7     group_size=128,
8     desc_act=False,
9     damp_percent=0.01
10 )
11
12 # Load model
13 model = AutoGPTQForCausalLM.from_pretrained(
14     "mistralai/Mistral-7B-v0.1",
15     quantize_config=quant_config,
16     max_memory={0: "10GiB", "cpu": "50GiB"}
17 )
18
19 # Quantize with calibration data
20 model.quantize(
21     calibration_data,
22     use_triton=False,
23     batch_size=1,
24     cache_examples_on_gpu=False
25 )
26
27 # Save quantized model
28 model.save_quantized(
29     "./mistral-7b-gptq-4bit",
30     use_safetensors=True
31 )

```

Optimized Inference:

For evaluation, we employ ExLlamaV2, an optimized inference engine providing $2\text{-}3\times$ speedup through specialized CUDA kernels for GPTQ models. ExLlamaV2 optimizations include custom kernels for 4-bit matrix multiplication, fused attention and MLP operations, and lazy KV cache loading.

1.4.3 AWQ (Activation-aware Weight Quantization)

AWQ protects salient weights based on activation magnitudes, preserving model accuracy through selective mixed-precision quantization [?]. The method identifies and protects the top 1% most influential weights while aggressively quantizing the remaining 99%.

Salient Weight Identification:

AWQ computes per-channel weight importance based on activation statistics. For weight matrix $\mathbf{W}$ and activation matrix $\mathbf{X}$, the importance of channel c is:

$$s_c = \|\mathbf{W}_{:,c}\|_2 \cdot \|\mathbf{X}_{c,:}\|_2 \quad (5)$$

Channels with highest s_c values are retained at higher precision (FP16) while others are quantized to 4-bit.

Per-channel Scaling:

AWQ applies learned per-channel scaling factors to balance activation magnitudes before quantization:

$$\mathbf{W}' = \mathbf{W} \cdot \text{diag}(\mathbf{s}), \quad \mathbf{X}' = \text{diag}(\mathbf{s}^{-1}) \cdot \mathbf{X} \quad (6)$$

where $\mathbf{s}$ are optimized to minimize quantization error while maintaining $\mathbf{W}'\mathbf{X}' = \mathbf{W}\mathbf{X}$.

Configuration:

- **Precision:** 4-bit integer weights, FP16 salient weights
- **Group size:** 128
- **Zero point:** Enabled (asymmetric quantization)
- **Version:** GEMM (optimized matrix multiplication kernels)
- **Salient weight ratio:** 1% (top channels protected)

Implementation:

We use pre-quantized AWQ models from TheBloke’s repository to avoid redundant quantization compute:

Listing 3: AWQ Model Loading

```

1  from awq import AutoAWQForCausalLM
2
3  model = AutoAWQForCausalLM.from_quantized(
4      "TheBloke/Mistral-7B-Instruct-v0.1-AWQ",
5      fuse_layers=True,
6      safetensors=True
7  )

```

1.4.4 HQQ (Half-Quadratic Quantization)

HQQ performs fast, calibration-free quantization by optimizing a custom half-quadratic loss function [?]. Unlike GPTQ and AWQ, HQQ requires no calibration data, making it suitable for rapid quantization without dataset access.

Half-Quadratic Optimization:

HQQ formulates quantization as minimizing:

$$\mathcal{L}(\mathbf{Q}, \mathbf{s}, \mathbf{z}) = \|\mathbf{W} - \mathbf{s} \odot \mathbf{Q} - \mathbf{z}\|_F^2 \quad (7)$$

where $\mathbf{Q}$ are quantized values, $\mathbf{s}$ are scales, $\mathbf{z}$ are zero-points, and $\odot$ denotes element-wise multiplication. The loss is optimized iteratively using a half-quadratic splitting technique that alternates between updating $\mathbf{Q}$ (quantization) and $(\mathbf{s}, \mathbf{z})$ (calibration parameters).

Row-wise Quantization:

HQQ applies quantization along weight matrix rows (axis=1), computing separate scales and zero-points for each output channel:

$$\hat{\mathbf{W}}_{i,:} = \text{quant}(\mathbf{W}_{i,:}, s_i, z_i) \quad \forall i \in [1, d_{\text{out}}] \quad (8)$$

This per-channel quantization preserves output distribution characteristics while avoiding the computational cost of Hessian-based methods.

Configuration:

- **Precision:** 4-bit integer
- **Group size:** 64
- **Quantization axis:** 1 (row-wise)
- **Compute dtype:** FP16
- **Optimization:** On-the-fly during model loading
- **Calibration:** None required

Implementation:

Listing 4: HQQ On-the-fly Quantization

```
1 from hqq.models.hf.base import AutoHQQHFModel
2 from hqq.core.quantize import BaseQuantizeConfig
3 import torch
4
5 # Configure quantization
6 quant_config = BaseQuantizeConfig(
7     nbits=4,
8     group_size=64,
9     axis=1 # Row-wise quantization
10 )
11
12 # Load and quantize model
13 model = AutoHQQHFModel.from_pretrained(
14     "mistralai/Mistral-7B-v0.1",
15     torch_dtype=torch.float16
16 )
17
18 model.quantize_model(
19     quant_config=quant_config,
20     device='cuda',
21 )
```

1.4.5 Quantization Summary

Table 1: Quantization methods comparison

Method	Calibration Required	Group Size	Optimization Basis	Quantization Time
NF4	No	64	Information theory	On-the-fly
GPTQ	Yes	128	Hessian (2nd-order)	20-25 min
AWQ	Yes	128	Activation-aware	50-60 min
HQQ	No	64	Half-quadratic	On-the-fly

All methods target 4-bit precision, achieving theoretical compression ratios of $3.6\text{-}4.0\times$ relative to FP16 baseline. NF4 and HQQ quantize on-the-fly during model loading, while GPTQ requires offline quantization on P100 GPUs. AWQ uses pre-quantized checkpoints from TheBloke’s repository for efficiency.

1.5 Calibration Dataset

For methods requiring calibration data (GPTQ and AWQ), we develop a custom RAG-specific calibration dataset that reflects realistic retrieval-augmented generation workloads. Unlike stan-

standard calibration approaches that use generic text corpora (e.g., C4, WikiText), our dataset is explicitly structured to match RAG prompt formats and complexity distributions.

1.5.1 Dataset Construction

We generate 128 calibration samples following the structured RAG format:

```
[QUERY]: {question}
[RETRIEVED DOCUMENTS]:
Document 1: {context_1}
Document 2: {context_2}
...
[ANSWER]: {answer}
```

This format mirrors production RAG systems where queries are paired with retrieved documents and expected answers, ensuring quantization parameters are optimized for the actual inference distribution.

1.5.2 Source Datasets

Calibration samples are synthesized from three complementary sources:

- **SQuAD v2** [?]: Provides high-quality question-answer pairs with clean, focused contexts for short-context samples
- **HotpotQA** [?]: Supplies multi-hop reasoning examples requiring information synthesis across multiple documents
- **Wikitext-103** [?]: Contributes general-domain passages as distractor documents and context padding

1.5.3 Distribution Strategy

We employ a stratified sampling approach targeting four complexity tiers that represent the operational envelope of RAG systems:

Table 2: RAG calibration dataset distribution

Category	Samples	Percentage	Token Range	Characteristics
Short-context	32	25%	256–512	Single document, direct extraction
Medium-context	40	31%	1024–2048	2–3 documents, comparison
Long-context	32	25%	3072–4096	5–7 documents, synthesis
Multi-hop	24	19%	Variable	8–10 documents, complex reasoning
Total	128	100%	256–4096	—

Short-context Samples (32 samples, 256–512 tokens) These samples simulate straightforward retrieval scenarios where a single relevant document contains the answer. We extract question-context-answer triples directly from SQuAD v2, representing the baseline RAG use case of locating information in focused passages.

Medium-context Samples (40 samples, 1024–2048 tokens) Medium-context samples introduce multi-document scenarios requiring comparison or selective attention. For each sample, we combine one relevant SQuAD context with 1–2 distractor passages from Wikitext-103, forcing the model to identify and focus on pertinent information among noise.

Long-context Samples (32 samples, 3072–4096 tokens) Long-context samples test the model’s ability to maintain coherence and extract information from extended contexts. We construct these by:

1. Selecting multi-hop questions from HotpotQA requiring reasoning across multiple facts
2. Combining 5–7 supporting context passages from HotpotQA and Wikitext-103
3. Shuffling document order to prevent positional bias
4. Padding to target length range with additional Wikipedia passages

Multi-hop Samples (24 samples, 8–10 documents) The most challenging category simulates complex retrieval scenarios requiring multi-step reasoning. We filter HotpotQA for comparison and bridge questions (requiring connecting information across documents), then construct contexts with 8–10 documents including both supporting and distractor passages. These samples stress-test the quantized model’s capacity for long-range dependencies and complex inference chains.

1.5.4 Dataset Statistics

The resulting calibration dataset exhibits the following statistical properties:

- **Total samples:** 128
- **Mean length:** 1,847 tokens (word count approximation)
- **Median length:** 1,523 tokens
- **Range:** 256–4,096 tokens
- **Standard deviation:** 1,124 tokens

Length distribution across bins:

- <512 tokens: 32 samples (25.0%)
- 512–1K tokens: 15 samples (11.7%)
- 1K–2K tokens: 41 samples (32.0%)
- 2K–4K tokens: 32 samples (25.0%)
- >4K tokens: 8 samples (6.3%)

This distribution ensures quantization algorithms observe representative activation patterns across the full spectrum of RAG workloads, from simple single-document retrieval to complex multi-hop reasoning over extensive contexts.

1.5.5 Calibration Protocol

For GPTQ quantization, we apply the following protocol:

- **Samples:** All 128 calibration samples
- **Sequence length:** Maximum 2048 tokens (truncated if necessary)
- **Batch size:** 1 (due to variable lengths)
- **Processing:** Sequential, no example caching on GPU

- **Hessian computation:** Accumulated across all calibration forward passes

For AWQ quantization, we use the same 128 samples with:

- **Sequence length:** Maximum 2048 tokens
- **Activation collection:** Per-channel statistics aggregated across all samples
- **Salient weight identification:** Top 1% channels protected based on activation magnitudes

By tailoring the calibration dataset to RAG-specific patterns, we ensure quantization algorithms preserve the activation distributions and weight sensitivities most critical for retrieval-augmented generation performance.

1.6 RAG Pipeline Configuration

1.6.1 Pipeline Architecture

Our RAG (Retrieval-Augmented Generation) pipeline implements a modular architecture consisting of five core components that work in sequence to enable context-aware question answering:

1. **Document Processing:** Extract and clean text from source documents (PDF, TXT, Markdown)
2. **Text Chunking:** Split documents into semantically coherent segments
3. **Embedding:** Convert text chunks into dense vector representations
4. **Vector Indexing:** Store and index embeddings for efficient similarity search
5. **Retrieval & Generation:** Query the index and generate contextualized answers

The pipeline is implemented as a reusable framework that can be applied to any of our compressed models, enabling direct comparison of RAG performance across quantization methods.

1.6.2 Document Processing

Text Extraction:

- **PDF Processing:** PyPDF2-based extraction with page-level granularity
- **Text Normalization:** Whitespace normalization, OCR error correction, quote standardization
- **Cleaning Operations:**
 - Remove page numbers and headers
 - Strip citation references ([1], (Author, 2020))
 - Remove URLs and excessive whitespace
 - Fix common ligature errors (fi, fl)

Table 3: Document processing parameters

Parameter	Default Value	Description
remove_headers	true	Strip page headers and numbers
remove_citations	true	Remove citation markers
extract_sections	false	Parse document sections

1.6.3 Text Chunking Strategy

We employ a **semantic chunking** strategy that respects natural document boundaries while maintaining optimal chunk sizes for embedding and retrieval. This approach outperforms fixed-size chunking by preserving contextual coherence.

Chunking Algorithm:

- **Strategy:** Semantic (paragraph-aware)
- **Primary Delimiter:** Double newlines (paragraph boundaries)
- **Fallback:** Sentence-level tokenization using NLTK punkt tokenizer
- **Overlap Mechanism:** Sliding window with configurable overlap to maintain context continuity across chunk boundaries

Table 4: Text chunking parameters

Parameter	Default Value	Description
strategy	semantic	Chunking strategy (semantic, sentence, fixed)
chunk_size	512	Target chunk size in tokens
chunk_overlap	50	Overlap between consecutive chunks
min_chunk_size	100	Minimum viable chunk size

Chunk Metadata: Each chunk includes metadata for traceability and filtering:

- **chunk_id:** Unique identifier (chunk_0, chunk_1, ...)
- **page_number:** Source page in original document
- **start_char / end_char:** Character offsets in source
- **tokens:** Word count for the chunk
- **section:** Optional section header (if extracted)

1.6.4 Embedding Model

We use **sentence-transformers/all-MiniLM-L6-v2** as our embedding model, balancing quality and efficiency for retrieval tasks.

Model Characteristics:

- **Architecture:** Distilled from Microsoft MiniLM
- **Embedding Dimension:** 384
- **Max Sequence Length:** 256 tokens
- **Training:** Fine-tuned on 1B+ sentence pairs
- **Performance:** SBERT benchmark score: 68.06 (semantic similarity)

Table 5: Embedding model parameters

Parameter	Default Value	Description
model_name	all-MiniLM-L6-v2	SentenceTransformer model identifier
batch_size	32	Batch size for embedding generation
normalize	true	L2 normalization of embeddings
device	cuda	Compute device (cuda, mps, cpu)

Implementation:

Listing 5: Embedding Generation

```
1 from sentence_transformers import SentenceTransformer
2
3 model = SentenceTransformer(
4     'sentence-transformers/all-MiniLM-L6-v2',
5     device='cuda'
6 )
7
8 embeddings = model.encode(
9     texts,
10    batch_size=32,
11    show_progress_bar=True,
12    normalize_embeddings=True,
13    convert_to_numpy=True
14 )
```

1.6.5 Vector Store and Indexing

We use **ChromaDB** as our vector database, providing efficient similarity search with multiple distance metrics.

Vector Store Configuration:

- **Database:** ChromaDB (persistent or in-memory)
- **Index Type:** HNSW (Hierarchical Navigable Small World)
- **Distance Metric:** Cosine similarity (default)
- **Storage:** Optional persistent storage for index reuse

Table 6: Vector store parameters

Parameter	Default Value	Description
collection_name	rag_documents	Index collection identifier
persist_directory	null	Directory for persistent storage (null=in-memory)
distance_metric	cosine	Distance function (cosine, l2, ip)

Distance-to-Similarity Conversion: ChromaDB returns distances that must be converted to similarity scores $[0, 1]$:

- **Cosine:** $\text{similarity} = 1 - \frac{d^2}{2}$ where d is L2 distance of normalized vectors
- **L2:** $\text{similarity} = \frac{1}{1+d}$ (exponential decay)
- **Inner Product:** $\text{similarity} = \frac{d+2}{2}$ (normalized to $[0, 1]$)

1.6.6 Retrieval Strategy

Our retrieval system implements advanced techniques beyond simple nearest-neighbor search:

Base Retrieval:

- **Top-K Selection:** Retrieve top-3 most similar chunks by default
- **Similarity Threshold:** Configurable minimum similarity score (default: 0.0)
- **Metadata Filtering:** Optional filtering by page number, section, etc.

Re-ranking (Optional): Hybrid retrieval combining semantic and lexical matching:

- **Semantic Score:** Original embedding similarity (70% weight)

- **Lexical Score:** Token overlap between query and chunk (30% weight)
- **Formula:** $\text{rerank_score} = 0.7 \times \text{cosine_sim} + 0.3 \times \text{token_overlap}$

Diversity Mechanism (Optional): Maximal Marginal Relevance (MMR) to reduce redundancy:

- **Objective:** Balance relevance and diversity in retrieved chunks
- **Formula:** $\text{MMR} = \lambda \times \text{Sim}(q, c) - (1 - \lambda) \times \max[\text{Sim}(c, S)]$
- where q is query, c is candidate chunk, S is selected chunks, λ is diversity parameter

Table 7: Retrieval parameters

Parameter	Default Value	Description
top_k	3	Number of chunks to retrieve
similarity_threshold	0.0	Minimum similarity score
rerank	false	Enable hybrid re-ranking
diversity_penalty	0.0	MMR diversity parameter [0, 1]

1.6.7 Answer Generation

The generation component uses the compressed LLM with retrieved context to produce grounded answers.

Prompt Engineering: We design prompts to encourage faithful, concise answers based on retrieved context:

Listing 6: RAG Generation Prompt Template

```

1 prompt = f"""Use the following context to answer the question.
2 Provide a clear, direct answer based on the information given.
3
4 Context:
5 {retrieved_context}
6
7 Question: {user_query}
8
9 Answer: ""

```

Generation Parameters: Carefully tuned to balance faithfulness and naturalness:

- **Max New Tokens:** 128 (concise answers)
- **Temperature:** 0.3 (low for factual accuracy)
- **Top-p:** 0.9 (nucleus sampling)
- **Repetition Penalty:** 1.15 (prevent loops)
- **Sampling:** Enabled (allows natural phrasing)

Table 8: Answer generation parameters

Parameter	Default Value	Description
max_new_tokens	128	Maximum answer length
temperature	0.3	Sampling temperature
top_p	0.9	Nucleus sampling threshold
do_sample	true	Enable sampling vs greedy
repetition_penalty	1.15	Penalty for repeated tokens
use_chat_template	true	Use model's chat template if available

Answer Validation: Post-processing to ensure quality:

- **Truncation:** Limit to 4 sentences maximum
- **Context Truncation:** Cap context at 2000 characters to prevent overwhelming
- **Retry Mechanism:** If answer appears problematic (too short, repetitive, verbatim copying), retry with simplified prompt and lower temperature (0.2)
- **Fallback:** Return "The information is not provided in the given context" for empty/invalid responses

1.6.8 RAG Evaluation Dataset

We create a custom technical QA dataset from documentation corpora to evaluate RAG performance:

Dataset Characteristics:

- **Domain:** Technical documentation (ML frameworks, APIs)
- **Size:** 10-50 question-answer pairs per evaluation
- **Question Types:**
 - Factual: "What is the default learning rate?"
 - Procedural: "How do you initialize a model?"
 - Comparative: "What's the difference between X and Y?"
- **Answer Format:** Short-form answers (1-3 sentences)
- **Ground Truth:** Human-verified reference answers

Table 9: RAG evaluation dataset parameters

Parameter	Default Value	Description
num_questions	10	Number of QA pairs to evaluate
dataset_path	null	Path to custom QA JSON file
compare_no_rag	true	Evaluate without retrieval baseline
save_detailed_responses	false	Save individual responses to file

Evaluation Protocol:

1. Index source documents using the RAG pipeline
2. For each test question:
 - Retrieve top-K relevant chunks
 - Generate answer with context (RAG)

- Generate answer without context (no-RAG baseline)
3. Compute metrics comparing predictions to reference answers
 4. Aggregate results across all questions

1.6.9 Pipeline Integration with Compressed Models

The RAG pipeline is model-agnostic and interfaces with any compressed model through a unified ModelInterface:

Listing 7: RAG Pipeline Initialization

```

1  from rag import RAGPipeline
2
3  # Load compressed model
4  model_interface = load_compressed_model("NF4")
5
6  # Initialize RAG pipeline
7  rag_config = {
8      "chunking": {"strategy": "semantic", "chunk_size": 512},
9      "embedding": {"model_name": "all-MiniLM-L6-v2"},
10     "retrieval": {"top_k": 3, "rerank": False},
11     "generation": {"temperature": 0.3, "max_new_tokens": 128}
12 }
13
14 pipeline = RAGPipeline(rag_config)
15 pipeline.setup(model_interface)
16
17 # Index documents
18 pipeline.index_documents("technical_docs.pdf")
19
20 # Evaluate
21 results = pipeline.evaluate(test_questions, compare_no_rag=True)

```

This design enables fair comparison of RAG performance across all compression methods, as all components except the generation model remain constant.

1.7 Evaluation Metrics and Benchmarks

We assess compressed models across three evaluation dimensions: computational efficiency, general task performance, and RAG-specific capabilities. All evaluations are conducted on the same Tesla T4 GPU to ensure fair comparison.

1.7.1 Computational Efficiency Metrics

Efficiency metrics quantify the speed, memory, and energy improvements achieved through quantization.

Table 10: Time performance metrics

Metric	Description
Latency (ms/token)	End-to-end generation speed measured as mean time per generated token across 10 inference runs with 3 warmup iterations
Time-to-First-Token (TTFT, ms)	Initial response latency measuring time from prompt submission to first token generation
Prefill Latency (ms)	Prompt processing time in parallel
Decode Latency (ms/token)	Autoregressive token generation time
Throughput (tokens/s)	Sustained generation rate: $\text{Throughput} = \frac{\sum_{i=1}^{10} \text{tokens}_i}{\sum_{i=1}^{10} \text{time}_i}$

Time Performance Metrics

Table 11: Space performance metrics

Metric	Description
Model Size (GB)	Disk storage requirements: $\text{Size}_{\text{GB}} = \frac{\sum(\text{element_size} \times \text{numel})}{1024^3}$
Peak Memory (MB)	Maximum GPU memory allocated during inference
Bits per Parameter	Average quantization level: $\text{Bits/param} = \frac{\text{Size}_{\text{bytes}} \times 8}{\text{Total parameters}}$
Compression Ratio	Size reduction relative to FP16: $\text{Compression} = \frac{\text{Size}_{\text{FP16}}}{\text{Size}_{\text{quantized}}}$
Memory Efficiency	Ratio of model size to peak memory: $\text{Efficiency} = \frac{\text{Model Size (GB)}}{\text{Peak Memory (GB)}}$
KV Cache Size (MB)	Attention cache memory for sequence length 2048: $\text{KV Size} = 2 \times L \times B \times H \times S \times D \times \text{bytes}$

Space Performance Metrics

1.7.2 General Task Performance Benchmarks

We evaluate compressed models on standard language modeling and reasoning benchmarks to assess general capability preservation.

Language Modeling: Perplexity Perplexity (PPL) measures next-token prediction quality on WikiText-2 test set:

$$\text{PPL}(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{<i}) \right) \quad (9)$$

Table 12: Perplexity evaluation parameters

Parameter	Value
Dataset	WikiText-2 test split
Samples	100 passages
Max length	512 tokens
Stride	512 (sliding window)
Min text length	100 characters

Core Task Benchmarks We evaluate models using the Language Model Evaluation Harness with standardized protocols. Our benchmark selection is organized into four categories:

Baseline Reasoning (0-shot):

- **HellaSwag**: Sentence completion requiring commonsense reasoning. Tests intuitive understanding preservation. Metric: normalized accuracy.
- **Winogrande**: Pronoun disambiguation requiring commonsense knowledge. Evaluates semantic understanding. Metric: accuracy.
- **PIQA**: Physical commonsense reasoning. Tests intuitive physics knowledge. Metric: normalized accuracy.

- **ARC-Easy**: Grade-school science questions. Assesses factual knowledge retention. Metric: normalized accuracy.
- **ARC-Challenge**: Challenge-level science questions. Tests complex multi-hop reasoning. Metric: normalized accuracy.

Core RAG Tasks (0-shot):

- **SQuAD**: Extractive question answering from Wikipedia paragraphs (1-2 paragraphs). Tests accurate answer extraction from short contexts. Metric: F1 score and exact match.
- **TriviaQA**: Open-domain QA with evidence documents. Tests robustness to longer, noisier contexts. Metric: exact match.

RAG with Complex Reasoning (3-shot):

- **DROP**: Discrete reasoning over paragraphs requiring arithmetic, counting, sorting. Tests numerical reasoning preservation. Metric: F1 score.
- **RACE**: Reading comprehension from educational exams requiring inference beyond surface matching. Tests complex comprehension. Metric: accuracy.

Fact Verification (0-shot):

- **BoolQ**: Yes/no question answering requiring statement verification against passages. Tests binary classification for hallucination detection. Metric: accuracy.

Table 13: Core task benchmark organization

Category	Tasks	Few-shot	Est. Time (min)	RAG Relevance
Baseline Reasoning	5	0	15	Establishes non-RAG baseline
Core RAG	2	0	20	Direct context QA
RAG + Reasoning	2	3	12	Complex multi-step inference
Verification	1	0	5	Hallucination detection
Total	10	—	52	—

All evaluations use greedy decoding (temperature=0) for reproducibility. Estimated times are per quantization method on Tesla T4 GPU.

1.7.3 RAG-Specific Performance Metrics

We develop custom evaluations to assess quantization impact on RAG-critical capabilities: attention preservation, context handling, and retrieval-augmented generation quality.

Attention Preservation This metric evaluates whether quantized models maintain proper attention to relevant documents in multi-document contexts.

Table 14: Attention preservation evaluation setup

Parameter	Value
Dataset	Natural Questions validation (500 samples)
Documents per query	5 (1 relevant + 4 distractors)
Document source	Wikipedia passages (≤ 250 chars)
Max context length	512 tokens
Generation length	20 tokens
Target samples	50 successful evaluations

Attention Extraction Protocol:

We extract document-level attention from model outputs during generation by:

1. Capturing attention weights from last transformer layer
2. Averaging across attention heads: $[B, H, S, S] \rightarrow [S, S]$
3. Extracting attention from last generated token to input: $[S, S] \rightarrow [S]$
4. Aggregating token-level attention to document level by equal division
5. Normalizing to probability distribution: $\sum_{i=1}^5 a_i = 1.0$

Attention snapshots are captured at three generation stages: start (token 1), middle (token $\lfloor N/2 \rfloor$), and end (token N), where N is the number of generated tokens.

Metrics:

Preservation Metrics:

- **Precision@1:** Fraction of queries where model assigns highest attention to relevant document. Range: $[0, 1]$, higher is better.
- **Mean/Median Rank:** Average rank of relevant document by attention weight. Range: $[1, 5]$, lower is better (1 = top-ranked).
- **Mean Attention on Answer:** Average attention weight on relevant document. Range: $[0, 1]$, higher indicates stronger focus.
- **Attention Concentration (Gini):** Distribution uniformity measured via Gini coefficient. Higher values indicate focused attention on fewer documents.

Attention Drift Metrics:

- **Mean Total Drift:** Average L1 distance between initial and final attention distributions:

$$\text{Drift} = \|\mathbf{a}_{\text{final}} - \mathbf{a}_{\text{initial}}\|_1 \quad (10)$$

where $\mathbf{a} \in \mathbb{R}^5$ is the document-level attention distribution. Range: $[0, 2]$, lower indicates more stable attention.

- **Mean Answer Drift:** Attention change on relevant document specifically:

$$\text{Answer Drift} = |a_{\text{relevant}}^{\text{final}} - a_{\text{relevant}}^{\text{initial}}| \quad (11)$$

Lower values indicate stable focus on relevant information during generation.

- **Mean Max Drift:** Maximum single-document attention change across all documents, indicating which document experienced largest attention shift.
- **Answer Drift Direction:** Signed difference indicating whether attention to answer document increased (+) or decreased (-) during generation.

Quality Correlation:

- **Exact Match:** Fraction of answers containing ground truth string (case-insensitive).
- **Mean F1:** Token-level F1 score between generated answers and ground truth, measuring whether attention preservation correlates with answer quality.

Context Length Degradation This metric evaluates performance degradation as context length increases, simulating long-document RAG scenarios.

Table 15: Context length degradation evaluation setup

Parameter	Value
Dataset	SQuAD v2 validation (300 samples)
Context lengths tested	[512, 1024, 2048, 4096] tokens
Filler passages	WikiText-103
Samples per length	25
Answer position	Middle of context (surrounded by filler)
Generation length	20 tokens

Context Assembly Protocol:

For each evaluation sample:

1. Start with question + answer-containing context (base passage from SQuAD)
2. Calculate token count of base passage
3. Add random Wikipedia filler passages until reaching target length
4. Shuffle filler to surround answer passage (placing it at target position)
5. Truncate to exact target length if exceeded
6. Append "Answer:" prompt

This ensures the answer is embedded in realistic long-context scenarios rather than isolated at the end, testing the model’s ability to locate and extract information from extended contexts with distractors.

Metrics:

Accuracy Metrics:

- **F1 by Length:** Token-level F1 score at each context length, measuring answer quality degradation
- **Accuracy by Length:** Exact match accuracy at each context length (binary correctness)

Degradation Analysis:

- **Slope per 1K tokens:** Linear regression slope of F1 vs. context length:

$$F1 = \beta_0 + \beta_1 \times \text{context_length} + \epsilon \quad (12)$$

Negative slopes indicate degradation, with steeper slopes showing worse robustness to long contexts. Interpretation:

- $|\beta_1| < 0.001$: negligible degradation
- $0.001 \leq |\beta_1| < 0.01$: minimal degradation

- $0.01 \leq |\beta_1| < 0.05$: moderate degradation
- $|\beta_1| \geq 0.05$: significant degradation
- **R-squared:** Regression goodness-of-fit, indicating whether degradation follows linear pattern. Values near 1.0 suggest consistent linear degradation; lower values indicate non-linear or irregular patterns.

Position Analysis:

- **Mean F1 by Position:** Average performance when answer is at start, middle, or end of context
- **F1 Standard Deviation:** Variability across different context lengths
- **Min/Max F1:** Performance range indicating robustness

RAG System Evaluation Complete evaluation of retrieval-augmented generation combining retrieval quality, answer generation quality, and system efficiency.

Table 16: RAG system evaluation setup

Parameter	Value
Test questions	10-50 Q&A pairs
Documents indexed	Technical documentation corpus
Retrieval method	Dense vector search (sentence-transformers)
Top-k chunks	3 per query
Max generation tokens	50
Compare no-RAG baseline	Yes

Metrics:

Retrieval Quality:

Table 17: Retrieval quality metrics

Metric	Description
Context Sufficiency	Fraction of queries where retrieved contexts contain answer (80% token overlap threshold)
Context Precision	Query-context relevance via token overlap
Answer Coverage	Fraction of answer tokens present in retrieved contexts
Avg Retrieval Score	Mean similarity score from vector search
Retrieval Consistency	Standard deviation of retrieval scores across queries
Avg Context Length	Mean word count of retrieved context

Answer Quality:

Table 18: Answer generation quality metrics

Metric	Description
Exact Match (EM)	Binary correctness: perfect normalized string match
F1 Score	Token-level precision-recall harmonic mean
Faithfulness	Fraction of answer tokens present in retrieved context (groundedness)
ROUGE-1/2/L	N-gram overlap (unigram, bigram) and longest common subsequence (optional)
Avg Answer Length	Mean word count of generated answers

RAG vs. No-RAG Comparison:

- **F1 Gain:** Absolute improvement: $\Delta F1 = F1_{\text{RAG}} - F1_{\text{no-RAG}}$
- **F1 Gain Percent:** Relative improvement: $\frac{\Delta F1}{F1_{\text{no-RAG}}} \times 100\%$
- **EM Gain:** Exact match improvement with retrieval augmentation

Efficiency:

Table 19: RAG system efficiency metrics

Metric	Description
Avg Retrieval Time (ms)	Mean time to retrieve and rank top-k chunks
Avg RAG Generation Time (ms)	Mean time to generate answer with context
Avg No-RAG Generation Time (ms)	Baseline generation time without context
RAG Overhead (ms)	Additional latency: $(t_{\text{retrieval}} + t_{\text{RAG-gen}}) - t_{\text{no-RAG-gen}}$
RAG Overhead Percent	Relative overhead: $\frac{\text{overhead}}{t_{\text{no-RAG-gen}}} \times 100\%$

All RAG evaluations are conducted on the same Tesla T4 GPU with identical retrieval configurations to ensure fair comparison across quantization methods. Retrieval uses cached embeddings to isolate generation performance from embedding computation overhead.

2 Results

2.1 LM-Eval Benchmarks

2.1.1 Baseline Reasoning Tasks

Table 20: Performance on Commonsense Reasoning Benchmarks

Model	Hellaswag	WinoGrande	PIQA	ARC-Easy	ARC-Challenge
FP16 Baseline	0.72	0.70	0.79	0.76	0.58
NF4	0.70	0.68	0.78	0.75	0.58
GPTQ	0.714	0.752	0.840	0.776	0.510
AWQ	—	—	—	—	—

2.1.2 Core RAG Tasks

Table 21: Reading Comprehension Performance (SQuADv2)

Model	Exact Match	F1	HasAns EM	HasAns F1	NoAns F1
FP16 Baseline	0.56	0.71	0.61	0.75	0.66
NF4	0.55	0.70	0.60	0.74	0.65
GPTQ	0.53	0.68	0.58	0.72	0.63
AWQ	—	—	—	—	—

2.1.3 RAG Reasoning Tasks

Table 22: Complex Reasoning over Context (DROP, 3-shot)

Model	Exact Match	F1	Date Understanding
FP16 Baseline	0.32	0.42	0.52
NF4	0.30	0.40	0.48
GPTQ	0.28	0.38	0.45
AWQ	—	—	—
HQQ	—	—	—

2.1.4 Additional Commonsense Tasks

Table 23: Commonsense QA

Model	CommonsenseQA Accuracy
FP16 Baseline	—
NF4	—
GPTQ	0.552
AWQ	—
HQQ	—

Table 24: Binary Fact Verification (BoolQ)

Model	Accuracy	Calibration Error
FP16 Baseline	0.79	0.08
NF4	0.77	0.10
GPTQ	0.75	0.11
AWQ	—	—
HQQ	—	—

2.2 Performance Metrics

2.2.1 Inference Performance

Table 25: Latency and Throughput Metrics

Model	Latency (ms/token)	Throughput (tokens/s)	TTFT (ms)
FP16 Baseline	64.92 ± 0.21	15.40 ± 0.05	71.70 ± 0.00
NF4	81.12 ± 0.44	12.30 ± 0.07	177.16 ± 4.02
GPTQ	21.32 ± 0.46	46.94 ± 1.03	46.68 ± 0.00
AWQ	78.68 ± 0.88	12.79 ± 0.03	79.25 ± 0.80

2.2.2 Memory Footprint

Table 26: Memory Usage Statistics

Model	Peak Memory (MB)	Allocated (MB)	Reserved (MB)	Model Size (GB)
FP16 Baseline	6930.64	6914.38	6940.00	13.49
NF4	1859.11	—	—	3.74
GPTQ	8098.41	8096.26	8130.00	3.12
AWQ	1869.04	—	—	3.87

2.3 Perplexity Evaluation

Table 27: Perplexity on Standard Datasets

Model	WikiText-2		UltraChat	
	PPL	Loss	PPL	Loss
FP16 Baseline	12.788	2.549	13.2	2.58
NF4	13.016	2.567	13.4	2.60
GPTQ	12.854	2.554	13.1	2.57
AWQ	13.474	2.601	13.8	2.62

Table 28: Perplexity Evaluation Metadata

Dataset	Samples	Total Tokens
WikiText-2	100	40,000–50,000
UltraChat	50	20,000–25,000

2.4 Context Length Degradation

2.4.1 Performance by Context Length

Table 29: F1 Score Degradation Across Context Lengths

Context Length	Start Position	Middle Position	End Position	Mean F1
<i>FP16 Baseline</i>				
512 tokens	0.20	0.17	0.20	0.19
1024 tokens	0.20	0.20	0.20	0.20
2048 tokens	0.20	0.20	0.20	0.20
4096 tokens	0.25	0.20	0.20	0.22
<i>NF4</i>				
512 tokens	0.78	0.74	0.76	0.76
1024 tokens	0.75	0.70	0.73	0.73
2048 tokens	0.71	0.66	0.69	0.69
4096 tokens	0.67	0.62	0.65	0.65
<i>GPTQ</i>				
512 tokens	0.12	0.04	0.07	0.08
1024 tokens	0.17	0.00	0.00	0.06
2048 tokens	—	—	—	—
4096 tokens	—	—	—	—
<i>AWQ</i>				
512 tokens	0.30	0.33	0.22	0.133760
1024 tokens	0.30	0.33	0.22	0.000000
2048 tokens	0.30	0.33	0.22	0.000000
4096 tokens	0.30	0.33	0.22	0.000000

2.4.2 Degradation Analysis

Table 30: Context Length Degradation Metrics

Metric	Value	Interpretation
<i>FP16 Baseline</i>		
Slope per 1K tokens	−0.007	Stable
R-squared	0.908	Strong linear fit
<i>NF4</i>		
Slope per 1K tokens	−0.037	Moderate degradation
R-squared	0.94	Strong linear fit
<i>GPTQ</i>		
Slope per 1K tokens	−0.039	Moderate degradation
R-squared	0.027	Weak linear fit
<i>AWQ</i>		
Slope per 1K tokens	0.000000	Performance gain/stable
R-squared	0.350725	Fit quality (Linear)

2.4.3 Position Sensitivity Summary

Table 31: Answer Position Sensitivity Statistics

Position	Mean F1	Std F1	Min F1	Max F1
<i>FP16 Baseline</i>				
Start	0.21	0.001	0.20	0.25
Middle	0.19	0.02	0.17	0.20
End	0.20	0.03	0.20	0.26
<i>NF4</i>				
Start	0.73	0.10	0.60	0.86
Middle	0.68	0.12	0.55	0.82
End	0.71	0.11	0.58	0.84
<i>GPTQ</i>				
Start	0.14	0.03	0.12	0.17
Middle	0.02	0.02	0.00	0.04
End	0.04	0.04	0.00	0.07
<i>AWQ</i>				
Start	0.283902	0.257486	0.000000	1.000000
Middle	0.339849	0.205990	0.083333	1.000000
End	0.227972	0.138104	0.000000	0.500000
<i>Exp. Start</i>	<i>0.72–0.80</i>	<i>0.08–0.12</i>	<i>0.55–0.65</i>	<i>0.85–0.92</i>
<i>Exp. Middle</i>	<i>0.65–0.75</i>	<i>0.10–0.15</i>	<i>0.48–0.58</i>	<i>0.80–0.88</i>
<i>Exp. End</i>	<i>0.68–0.78</i>	<i>0.09–0.13</i>	<i>0.52–0.62</i>	<i>0.82–0.90</i>

2.5 RAG System Evaluation

2.5.1 Retrieval Quality

Table 32: Retrieval Quality Metrics on SQuADv2

Model	Context Sufficiency	Answer Coverage	Avg Retrieval Score	Avg Context Length (words)
FP16 Baseline	0.701	0.716	0.262	1237.1
NF4	0.796	0.756	0.800	1308.45
GPTQ	0.223	0.705	0.262	1215.3
AWQ	0.043956	0.455000	N/A (Dense)	111.280000

2.5.2 Answer Quality

Table 33: Answer Quality with and without RAG

Model	Exact Match	F1 Score	Faithfulness	ROUGE-1	ROUGE-L
<i>With RAG</i>					
FP16 Baseline	0.003	0.184	0.421	0.165	0.157
NF4	0.00	0.205	0.539	0.244	0.177
GPTQ	0.00	0.081	0.432	0.215	0.162
AWQ	0.020000	0.133760	0.043956	0.125280	0.125054
<i>Without RAG (Baseline)</i>					
FP16 Baseline	0.00	0.068	—	0.065	—
NF4	0.00	0.181	—	—	—
GPTQ	0.00	0.084	—	—	—
AWQ	0.000000	0.009705	—	0.009332	0.008676
<i>Exp. w/ RAG</i>	<i>0.55–0.65</i>	<i>0.70–0.80</i>	<i>0.75–0.85</i>	<i>0.60–0.70</i>	<i>0.55–0.65</i>
<i>Exp. w/o RAG</i>	<i>0.20–0.35</i>	<i>0.30–0.45</i>	<i>-</i>	<i>0.25–0.40</i>	<i>0.22–0.37</i>

2.5.3 RAG Improvement

Table 34: RAG vs No-RAG Performance Gains

Model	F1 Gain	F1 Gain (%)	EM Gain
FP16 Baseline	0.116	169.8%	0.003
NF4	0.024	13.2%	0.00
GPTQ	-0.003	-4.1%	0.00
AWQ	0.123733	1671.5%	0.020000

2.5.4 Efficiency Metrics

Table 35: RAG System Efficiency

Model	Retrieval Time (ms)	RAG Gen Time (ms)	No-RAG Gen Time (ms)	RAG Overhead (%)
FP16 Baseline	7.93	1994.73	1902.74	5.3%
NF4	27.97	10443.94	9964.96	+4.8%
GPTQ	25.50	8850.0	8200.0	+7.9%
AWQ	0.00	3618.89	7895.15	-54.2%

2.5.5 RAG Configuration

Table 36: RAG System Configuration and Metadata

Parameter	Value
Corpus	SQuADv2
Documents Indexed	500
Test Questions	45
Retrieval Top-K	3
Chunk Size	512 tokens
Chunk Overlap	50 tokens
Embedding Model	sentence-transformers/all-MiniLM-L6-v2
Chunking Strategy	semantic
Max New Tokens	64
Temperature	0.3
Avg Chunks per Query	3.0

2.6 Summary Comparison

Table 37: Comprehensive Performance Summary - FP16 Baseline

Evaluation Category	Key Metric	Value
LM-Eval Benchmarks		
Reasoning (avg 4 tasks)	Accuracy	0.528
SQuAD Reading	F1 Score	0.71
DROP Reasoning	F1 Score	0.42
BoolQ Verification	Accuracy	0.79
Performance		
Inference	Throughput (tokens/s)	15.40
Latency	ms/token	64.92
Memory	Peak (MB)	6930.64
Model Size	GB	13.49
Language Modeling		
WikiText-2	Perplexity	8.14
UltraChat	Perplexity	7.56
Context Length		
Degradation	Slope per 1K tokens	−0.065
1024 tokens	Mean F1	0.06
RAG System		
Answer Quality	F1 Score	0.058
Retrieval Quality	Context Sufficiency	0.223
RAG Improvement	F1 Gain (%)	−28.9%
Faithfulness	Token Overlap	0.332

2.7 Notes and Interpretations

Metric Interpretations

Context Length Degradation

- **Negligible:** $|slope| < 0.001$ per 1K tokens
- **Minimal:** $0.001 \leq |slope| < 0.01$ per 1K tokens
- **Moderate:** $0.01 \leq |slope| < 0.05$ per 1K tokens
- **Significant:** $|slope| \geq 0.05$ per 1K tokens

Faithfulness Scores

- **High Faithfulness:** ≥ 0.80 (answers well-grounded in context)
- **Moderate Faithfulness:** 0.60–0.80 (some hallucination)
- **Low Faithfulness:** < 0.60 (significant hallucination)

RAG Improvement

- Positive F1 gain indicates RAG provides useful context
- Typical gains: 80–120% improvement over no-RAG baseline
- Overhead: 20–40% additional latency for retrieval + context

Dataset Details

- **WikiText-2**: 100 samples, 1024 max tokens, validation split
- **UltraChat (C4)**: 100 samples, 1024 max tokens, train_sft split
- **SQuAD v2**: 45 questions, 500 documents indexed
- **Context Length**: 25 samples per length, positions: start/middle/end

Generation Settings

- **Performance**: greedy decoding, 128 max tokens, 10 runs + 3 warmup
- **Context Length**: greedy decoding, 20 max tokens
- **RAG**: temperature 0.3, top-p 0.9, 64 max tokens, repetition penalty 1.15